

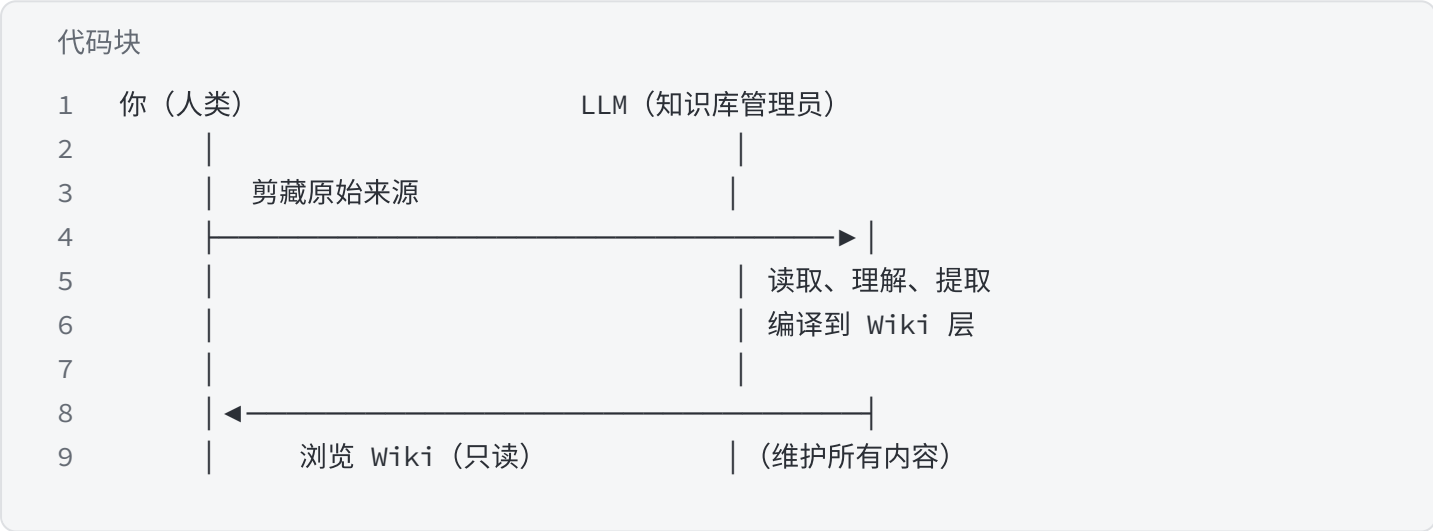
LLM Wiki 搭建教程

LLM Wiki 搭建教程

基于 Andrej Karpathy llm-wiki 模式的个人知识库系统。
核心理念：你只负责剪裁，LLM 负责理解和沉淀。

核心思想

与传统 RAG 的根本区别在于，它将知识「编译一次、持续维护」，而非「每次查询时重新推导」。



核心原则

1. Raw 层不可变 — 原始来源绝对不修改
2. Wiki 层 LLM 完全拥有 — 你只浏览，不编辑
3. 输出必须持久化 — 答案写入 outputs/，不消失在对话中
4. 矛盾必须显式标注 — 来源分歧明确记录
5. 每次操作都记日志 — 所有操作写入 wiki/log.md

核心架构分层

层级	归属者	内容	不可违反的规则
Raw 原始层	你（人类）	剪藏文章、PDF、图片、笔记	IMMUTABLE，LLM 绝不修改
Wiki 编译层	LLM 完全拥有	Markdown 实体/概念/来源/合成页	全部由 LLM 写入，人类只读浏览
Outputs层	LLM 生成	Q&A 答案、图表、Marp 幻灯片	输出必须回写入 Wiki，不消失于对话历史
CLAUDE.md	LLM 生成并维护	Wiki 结构约定与行为规则	精简不冗余，随使用持续迭代

文件目录设计

代码块

```
1  knowledge-base/
2  |—— raw/                                # 人类所有，LLM 只读
3  |   |—— articles/                       # 手动保存的文章（Markdown）
4  |   |—— clippings/                     # Obsidian Web Clipper 剪藏（主要入口）
5  |   |—— images/                         # 截图和图片
6  |   |—— pdfs/                           # PDF 文件及配套元数据文件
7  |   |—— notes/                           # 随手记录
8  |   |—— personal/                       # ★ 自己写的文章、分析报告、投资笔记
9  |—— wiki/                               # LLM 完全拥有
10 |   |—— index.md                         # LLM 读取的第一个文件，面向内容的索引
11 |   |—— log.md                           # 仅追加的操作日志（含 graph-excluded: true）
12 |   |—— overview.md                     # 高层综述 + Health Dashboard
13 |   |—— QUESTIONS.md                   # 开放问题队列
14 |   |—— sources/                         # 每个来源的摘要页
15 |   |—— concepts/                       # 思想、模式、技术（含 aliases 跨语言字段）
16 |   |—— entities/                       # 人物、工具、机构、论文
17 |   |—— synthesis/                     # 跨来源合成分析
18 |   |—— templates/                     # 页面模板（LLM 使用）
19 |—— outputs/                             # 查询答案、图表、幻灯片、lint 报告
20 |   |—— lint.md                           # lint 报告
21 |   |—— query.md                         # 查询答案
22 |—— scripts/
23 |   |—— lint.py                           # Wiki 健康检查脚本（9 项检查）
24 |   |—— qmd-reindex.sh                   # qmd 索引重建脚本
25 |—— CLAUDE.md                             # LLM 行为契约（核心文件）
26 |—— README.md                           # 项目说明
```

推荐插件

插件	用途	安装方式
Obsidian Web Clipper	浏览器剪裁到 raw/	浏览器扩展商店
Dataview	查询 frontmatter, 显示统计	Obsidian 社区插件
Marp for Obsidian	预览 LLM 生成的幻灯片	Obsidian 社区插件
Graph View (内置)	可视化 Wiki 概念关系, 建议开启 Node name: Title 显示中文	无需安装

搭建步骤

安装环境

- Node.js (用于安装各种软件, <https://nodejs.org/en>)
- 已安装 Claude Code (`npm install -g @anthropic-ai/claude-code`)
- Python 3.8+ (用于 lint 脚本)
- Obsidian (<https://obsidian.md/>, 用于管理本地化知识库)
- qmd 已安装 (`npm install -g @tobi/qmd`)

Step 1: Bootstrap (基础搭建)

代码块

```
1  mkdir -p ~/knowledge-base
2
3  cd ~/knowledge-base
4
5  在终端输入: claude code
```

将以下提示词完整粘贴给 Claude Code：

代码块

```
1 请帮我从零搭建一个基于 Karpathy LLM Wiki 思路的个人知识库系统。
2 完整执行以下所有步骤，不要遗漏任何细节。
3
4 ## 一、创建目录结构
5
6 创建以下目录：
7 raw/articles/
8 raw/clippings/
9 raw/images/
10 raw/pdfs/
11 raw/notes/
12 raw/personal/
13 wiki/sources/
14 wiki/concepts/
15 wiki/entities/
16 wiki/synthesis/
17 wiki/templates/
18 outputs/
19 scripts/
20
21 ## 二、创建系统文件
22
23 ### wiki/index.md
24 frontmatter 包含: type: system-index, graph-excluded: true
25 正文包含: Sources 列表（按日期倒序）、Concepts 列表、Entities 列表、Recent
    Synthesis 列表、Outputs 列表
26
27 ### wiki/log.md
28 frontmatter 包含: type: system-log, graph-excluded: true
29 说明: 仅追加操作日志，格式为「YYYY-MM-DD | 操作类型 | 说明」
30
31 ### wiki/overview.md
32 frontmatter 包含: type: system-overview, graph-excluded: true
33 包含: Knowledge Base Health Dashboard 表格（总来源数、高置信度概念数、开放问题数、
    Stale 页面数）
34
35 ### wiki/QUESTIONS.md
36 frontmatter 包含: type: system-questions, graph-excluded: true
37 包含: Open Questions 列表（checkbox 格式）、Resolved Questions 列表
38
39 ## 三、创建页面模板
40
41 ### wiki/templates/source-template.md
42 frontmatter 字段: type, title, date, source_url, domain, author, tags,
    processed, raw_file, raw_sha256, last_verified, possibly_outdated, language,
```

canonical_source

正文结构: ## Summary、## Key Points、## Concepts Extracted、## Entities
Extracted、## Contradictions (与其他来源的分歧)、## My Notes

wiki/templates/personal-writing-template.md

frontmatter 字段: type: personal-writing, title, date,
status(draft/published/deprecated), topic_tags,
confidence_at_writing(low/medium/high), superseded_by, raw_file, raw_sha256,
last_verified, tags, processed

正文结构: ## Core Argument、## Key Claims、## Evidence Referenced、## Limitations

wiki/templates/concept-template.md

frontmatter 字段: type: concept, title (中文主名称), date, updated, tags,
source_count, confidence(low/medium/high), domain_volatility(low/medium/high),
last_reviewed, aliases (数组, 存储中英文所有叫法)

正文结构: ## Definition (首行用「中文名 (English Name)」格式)、## Key Points、##
My Position、## Contradictions、## Sources (仅 wikilinks 列表)、## Evolution Log
(每次更新追加一条)

wiki/templates/entity-template.md

frontmatter 字段: type: entity, title, date, tags,
entity_type(person/tool/institution/paper), aliases

正文结构: ## Description、## Key Contributions、## Related Concepts、## Sources

wiki/templates/synthesis-template.md

frontmatter 字段: type: synthesis, title, date, tags, source_count, confidence

正文结构: ## Thesis、## Evidence、## Counter-evidence (Stage 0 反向检验结果)、##
Synthesis、## Confidence Notes、## Limitations、## Sources

四、创建 scripts/lint.py

lint.py 执行以下 9 项检查, 完成后将报告写入 wiki/outputs/lint-YYYY-MM-DD.md
(frontmatter 含 graph-excluded: true):

1. YAML frontmatter 合法性: 所有 wiki/ 下的 .md 文件是否有合法 YAML frontmatter
(含 type 和 date)

2. Broken Wikilinks: [[xxx]] 引用了不存在的页面

3. Index 一致性: wiki/index.md 中标记的文件是否都实际存在

4. Stub 页面: 正文少于 100 字的空壳页面

5. 近重复概念名称: slug 名称 Jaccard 相似度 > 0.7 的 concept 页对

6. SHA-256 完整性: raw 文件哈希与 source 页 raw_sha256 字段比对 (△ SOURCE
MODIFIED)

7. Stale 页面: 超过 domain_volatility 时效阈值 (high=90天, medium=180天, low=365
天)

8. 跨语言重复: source URL 相似度检测 + 不同 concept 页的 aliases 字段重叠检测

9. Wikilink 格式规范: 检测非英文小写连字符格式的 wikilink (如中文词汇 [[价值投资]]) 及
别名断链

```
74
75  ## 五、创建 CLAUDE.md (行为契约)
76
77  CLAUDE.md 是 LLM 的核心行为规范，必须包含以下所有章节：
78
79  ### 系统概述
80  - 三层架构说明 (Raw/Wiki/Outputs)
81  - 核心原则：你完全拥有 wiki/ 目录的读取和写入权限，raw/ 目录由我（人类）拥有，你只能读
    取，绝不修改。
82
83  ### INGEST 操作规范
84  触发词：ingest、摄入、处理这个
85
86  来源类型判断（优先级由高到低）：
87  1. frontmatter 含 type: personal-writing → 走「个人写作」流程
88  2. 文件路径包含 raw/personal/ → 走「个人写作」流程
89  3. frontmatter 含 type: pdf-reference → 走「PDF 参考」流程
90  4. 其他 → 走「外部来源」标准流程
91
92  缺少 frontmatter 时的处理规则：
93  - 从文件第一个 # 标题提取 title；若无标题则从文件名推断
94  - source 字段留空，在 wiki/sources/<slug>.md 中标注「来源未知」
95  - date 使用文件系统修改时间
96  - 不中断 INGEST，但在 log.md 记录「警告：来源文件缺少标准 frontmatter」
97
98  **外部来源标准流程（11 步）**：
99  1. 读取目标原始来源 (raw/ 中的文件，只读)
100  2. 计算原始文件的 SHA-256 哈希 (Python hashlib)
101  3. 与用户确认核心要点（逐一摄入，保持参与感）
102  4. 生成 slug (小写英文，用连字符，例如 `attention-is-all-you-need`)
103  5. 创建 wiki/sources/<slug>.md (使用 source-template.md)，frontmatter 中写入：
104      - `raw_file`：相对路径 (如 `raw/articles/filename.md`)
105      - `raw_sha256`：SHA-256 哈希值
106      - `last_verified`：摄入日期 (YYYY-MM-DD)
107      - 若来源发表日期超过 2 年前：标注 `possibly_outdated: true`，并在摘要末尾添加提示
108  6. **概念名称对齐检查** (提取概念之前必须执行)：
109      - 将每个提取到的概念名称统一映射为英文小写连字符 slug (例如「第一性原理」→「first-
    principles-thinking」)
110      - 在 wiki/concepts/ 中查找该 slug 是否已存在对应文件
111      - **同时检查所有已有 concept 页的 `aliases` 字段**：遍历 wiki/concepts/*.md，解析
    每页 frontmatter 的 aliases 列表，检查是否包含当前概念名称 (支持中英文别名匹配)
112      - 若通过 slug 匹配或通过 aliases 匹配到已有页面：更新已有页面，不创建新页面
113      - 若找不到任何匹配：才创建新页面，并在 frontmatter 的 `aliases` 中同时填入中文名和
    英文名 (如果有的话)
114  7. 为每个提取到的概念：
115      - 如果 wiki/concepts/<concept>.md 已存在：更新它，追加新来源引用，在 Evolution
    Log 追加记录，更新 source_count 和 confidence，**同时更新 last_reviewed 字段**
```

116 - 如果不存在：创建新文件（使用 concept-template.md），**同时在 aliases 字段填入该
概念的中英文名称**

117 - **Evolution Log 追加规则**：

118 - 若本次来源与当前 Definition 一致：写「强化」

119 - 若有修正：写「修正：[具体变化]」

120 - 若相互矛盾：写「新增分歧：[分歧内容]，见 Contradictions 节」

121 - 格式：`- YYYY-MM-DD (N sources) : [本次认知变化的一句话描述]`

122 8. 为每个提取到的实体：同上逻辑

123 9. 更新 wiki/index.md：将来源从 Unprocessed 移动到 Processed

124 10. 读取 wiki/QUESTIONS.md，检查本次来源是否能回答开放问题：

125 - 若能：提示用户「此来源可能回答了开放问题：[问题描述]，是否立即执行 QUERY？」

126 - 用户确认后，执行 QUERY 并将结果写入 wiki/synthesis/，同时在 QUESTIONS.md 中将该
问题移入 Answered

127 11. 在 wiki/log.md 末尾追加：`YYYY-MM-DD HH:MM | ingest | [来源标题]`

128

129 **个人写作流程（不同于标准流程）**：

130 - 不生成 Summary 节，跳过客观摘要

131 - 核心论点写入相关 concept 页的 ## My Position 节（标注「个人认知」）

132 - 不参与 confidence 的 source_count 计数（避免用自己的文章给自己背书）

133 - 若文章中引用了外部来源，提取这些引用并尝试与已有 wiki/sources/ 页面建立 wikilinks

134 - raw_sha256 哈希机制同样适用

135 - Evolution Log 记录：「YYYY-MM-DD 个人写作 [[slug]] 确立了对此概念的明确立场」

136

137 ### QUERY 操作规范

138 触发词：直接提问，或「根据我的知识库」

139

140 执行步骤：

141 Step Q1: 执行 qmd query "<用户问题>" --json，获取 top 5 相关页面（若 qmd 报错则降级读
取 wiki/index.md）

142 Step Q2: 逐一完整读取 top 5 文件

143 Step Q3: 合成答案，每个核心结论必须溯源到具体 wiki/sources/<slug>.md（不允许只引用
concept 页）；注明各来源 confidence 级别；来源相互矛盾时显式标注分歧

144 Step Q4: 若答案具有复用价值，写入 wiki/outputs/YYYY-MM-DD-<topic>.md，文件
frontmatter 含 graph-excluded: true；在输出末尾包含「⚠ Confidence Notes」节；更新
wiki/index.md 的 Recent Synthesis 列表；追加 wiki/log.md

145

146 输出格式按问题类型：

147 - 普通问题 → Markdown 正文

148 - 比较类 → Markdown 表格

149 - 演示类 → Marp 幻灯片（frontmatter 加 marp: true）

150 - 趋势类 → Python matplotlib 代码块

151 - 清单类 → 结构化 bullet list

152

153 ### LINT 操作规范

154 触发词：lint、检查、健康检查

155

156 执行步骤：

157 1. 运行 scripts/lint.py (包含 9 项检查)

158 2. 将报告写入 wiki/outputs/lint-YYYY-MM-DD.md (frontmatter 含 graph-excluded: true)

159 3. 执行 qmd status, 对比索引文件数与 wiki/ 实际 .md 文件数 (排除系统文件); 若索引落后则执行 qmd add wiki/, 在报告中记录

160 4. 向用户展示摘要并询问是否修复

161

162 ### REFLECT 操作规范

163 触发词: reflect、综合分析、发现规律

164

165 四阶段执行:

166 Stage 0 (反向检验): 在生成任何合成结论之前, 主动搜索反驳证据。若无反对来源, 在 Limitations 节标注「⚠ 回音室风险: 未找到反驳来源, 结论可能存在确认偏差」

167 Stage 1 (模式扫描): 使用 qmd 批量扫描

168 qmd multi-get "wiki/concepts/*.md" -l 40

169 qmd multi-get "wiki/entities/*.md" -l 40

170 qmd multi-get "wiki/synthesis/*.md" -l 60

171 识别跨来源模式、隐性关联、内容空白、矛盾对

172 Stage 2 (深度合成): 对有证据支撑的候选项, 完整读取相关页面, 写入 wiki/synthesis/<topic>-synthesis.md

173 Stage 3 (Gap Analysis):

174 - source_count = 1 且创建超过 30 天的孤立概念

175 - 多处提及但无独立页面的概念/实体 (隐性盲区)

176 - 覆盖明显稀薄的主题领域

177 - 输出到 wiki/outputs/gap-report-YYYY-MM-DD.md (frontmatter 含 graph-excluded: true)

178

179 完成后更新 wiki/overview.md 的 Health Dashboard, 更新 wiki/index.md, 追加 wiki/log.md

180

181 ### MERGE 操作规范

182 触发词: merge、去重

183

184 同语言合并流程:

185 1. 与用户确认合并方案 (绝不自动合并)

186 2. 主 slug 保留, 被合并页面的 wikilinks 全部更新

187 3. 被合并文件替换为重定向文件 (内容: redirect: [[wiki/concepts/主slug]])

188 4. log.md 记录: YYYY-MM-DD | merge | [旧slug] → [主slug]

189

190 跨语言合并专项流程 (区别于同语言 MERGE):

191 1. 主 slug 保留英文

192 2. aliases 取两个页面的并集

193 3. Key Points / Sources / Evolution Log 按并集+去重合并

194 4. My Position 若两页都有, 先向用户展示对比后再合并

195 5. 被合并的旧 slug 文件保留为 redirect 文件 (确保旧 wikilinks 不 broken)

196 6. log.md 记录: YYYY-MM-DD | merge | [旧slug] → [主slug] (跨语言合并)

197

198 ### ADD-QUESTION 操作规范

199 触发词：我想搞清楚、add question、记录一个问题

200

201 执行步骤：

202 1. 将问题规范化（提取核心疑问）

203 2. 追加到 wiki/QUESTIONS.md (checkbox 格式：- [] 问题内容 (opened YYYY-MM-DD))

204 3. 追加 wiki/log.md

205

206 ### Wikilink 使用规范

207

208 **格式铁律（不可违反）**：

209 所有 wikilink 目标必须使用英文小写连字符格式

210  `[[value-investing]]`  `[[attention-mechanism]]`  `[[warren-buffett]]`

211  `[[价值投资]]`（中文词汇）  `[[ValueInvesting]]`（驼峰）  `[[value_investing]]`（下划线）

212

213 中文名称的正确处理方式：

214 - 写入 concept 页 frontmatter 的 aliases 字段

215 - concept 页正文第一行使用括号标注：「价值投资 (Value Investing)」

216 - wikilink 始终用英文 slug

217

218 **允许使用 wikilinks 的场景**：

219 - concept 页引用其他 concept/entity 页

220 - source 页引用 concept/entity 页

221 - synthesis 页引用 concept/source/entity 页

222

223 **禁止使用 wikilinks 的场景**：

224 - 任何页面不得引用系统文件：[[log]] [[index]] [[overview]] [[QUESTIONS]]

225 - 任何页面不得引用 lint 报告：[[outputs/lint-xxx]]

226 - 任何页面不得以操作名称作为 wikilink: [[ingest]] [[query]] [[reflect]]

227 - log.md 内部记录使用纯文本路径（如 wiki/sources/xxx.md），不使用 wikilinks

228

229 ### Wiki 语言规范

230 - Wiki 层 (concept/entity/synthesis 页) 统一用中文写作

231 - concept 页 title 字段使用中文主名称（图谱节点显示）

232 - 英文术语在 concept 页首次出现时括号标注：「注意力机制 (Attention Mechanism)」

233 - 所有 slug（文件名）统一用英文小写连字符，不使用中文文件名

234 - aliases 字段覆盖中英文所有叫法

235

236 ### Confidence 更新规则

237 | 来源数量 | Confidence | 处理方式 |

238 | ---|---|---|

239 | 1 个来源 | low | 自动设置 |

240 | 3+ 个来源 | medium | 自动设置 |

241 | 5+ 个来源且无重大矛盾 | 候选 high | 向用户展示 Definition 和 Sources 列表，等待确认 |

242 | 用户明确回复「确认」或「ok」 | high | 才可设置 |

```
243
244 注意：个人写作 (raw/personal/) 不参与 source_count 计数
245
246 ### Source Integrity Rules
247 - re-ingest 规则：若 lint 报告 ⚠ SOURCE MODIFIED，需重新摄入该文件并更新所有受影响的
  concept/entity 页面，Evolution Log 记录「YYYY-MM-DD 来源更新：[[slug]] 哈希变更，内容已重新提取」
248 - 来源超过 2 年标注 possibly_outdated: true
249 - 矛盾来源必须在 source 页和 concept 页的 Contradictions 节显式记录，不得静默覆盖
250
251 ### 系统文件隔离规则
252 以下文件的 frontmatter 必须含 graph-excluded: true，不参与 Obsidian 图谱：
253 - wiki/log.md
254 - wiki/index.md
255 - wiki/overview.md
256 - wiki/QUESTIONS.md
257 - wiki/outputs/ 下所有文件
258
259 ### 文档维护规则
260 当 CLAUDE.md 规则更新时，同步更新 USER_GUIDE.md 对应章节，确保两份文档一致。
261
262 ## 六、初始化 qmd 索引
263
264 执行：
265 qmd add wiki/
266 qmd status
267
268 ## 八、执行完成后的验证
269
270 输出以下验证报告：
271 1. 目录结构树 (tree -L 3 或 find)
272 2. CLAUDE.md 包含的章节列表
273 3. wiki/templates/ 下的模板文件列表
274 4. qmd status 输出 (索引文件数量)
275 5. scripts/lint.py 包含的检查项列表
```

Step 2: 标定（首次使用前必做）

Karpathy 原文特别强调：在正式批量处理前，先交互式标定 2-3 篇最典型的来源，审查输出质量后再调整 CLAUDE.md。

代码块

```
1 # 先放入 1 篇测试文章
2 cp ~/你的文章.md ~/knowledge-base/raw/articles/
```

```
3
4 # 告诉 Claude Code
5 你: ingest raw/articles/你的文章.md
6 # 仔细审查: 摘要质量、概念提取是否准确、wikilinks 格式是否正确
7 # 若不满意, 告诉 LLM: 请更新 CLAUDE.md, 以后处理 [情况] 时要 [规范]
```

标定后再处理剩余文章, 避免大量文章风格不一致。

Step 3: 全系统 Audit

搭建完成后, 执行一次完整的系统状态核查, 确认所有规则已落地:

代码块

```
1  请对当前知识库做一次完整的系统状态核查, 逐项验证以下内容,
2  输出核查报告到 wiki/outputs/system-audit-YYYY-MM-DD.md:
3
4  ## 一、目录结构完整性
5  验证 raw/ 和 wiki/ 下所有子目录是否存在
6
7  ## 二、CLAUDE.md 关键规则覆盖 (逐项输出是/否)
8  - [ ] Raw 层不可变原则
9  - [ ] INGEST 来源类型判断 (personal-writing vs 外部来源)
10 - [ ] INGEST SHA-256 哈希记录规则
11 - [ ] INGEST 去重检测 (含 canonical_source 译文检测)
12 - [ ] INGEST 概念名称对齐检查 (aliases 匹配)
13 - [ ] INGEST QUESTIONS.md 匹配检查
14 - [ ] INGEST 缺少 frontmatter 的处理规则
15 - [ ] INGEST URL 直接输入的 defuddle 调用规则
16 - [ ] INGEST 最后一步执行 qmd update
17 - [ ] QUERY 使用 qmd query 优先 (含 fallback)
18 - [ ] QUERY 来源溯源要求 (追溯到 sources 页)
19 - [ ] QUERY Confidence Notes 输出要求
20 - [ ] QUERY 高价值答案持久化规则
21 - [ ] confidence: high 必须用户确认, 禁止自动晋升
22 - [ ] LINT 运行 scripts/lint.py (9 项检查)
23 - [ ] LINT 执行 qmd 索引同步验证
24 - [ ] REFLECT Stage 0 反向检验
25 - [ ] REFLECT Stage 1 使用 qmd multi-get 批量扫描
26 - [ ] REFLECT Stage 3 Gap Analysis
27 - [ ] MERGE 跨语言合并专项流程 (redirect 文件保留)
28 - [ ] Wikilink 格式铁律 (英文小写连字符)
29 - [ ] Wikilink 禁止清单 (系统文件不得被 wikilink)
30 - [ ] Wiki 语言规范 (中文写作, 英文 slug, aliases 跨语言)
31 - [ ] 系统文件隔离规则 (graph-excluded: true)
```

```
32 - [ ] 文档维护规则 (CLAUDE.md 更新时同步 USER_GUIDE.md)
33
34 ## 三、模板文件完整性 (逐项验证必需字段)
35 - [ ] source-template.md 含 language / canonical_source
36 - [ ] personal-writing-template.md 含 type: personal-writing /
confidence_at_writing
37 - [ ] concept-template.md 含 aliases / domain_volatility / last_reviewed /
Evolution Log
38 - [ ] entity-template.md 含 aliases
39 - [ ] synthesis-template.md 含 Counter-evidence / Confidence Notes /
Limitations
40
41 ## 四、系统文件隔离状态
42 - [ ] wiki/log.md 含 graph-excluded: true
43 - [ ] wiki/index.md 含 graph-excluded: true
44 - [ ] wiki/overview.md 含 graph-excluded: true
45 - [ ] wiki/QUESTIONS.md 含 graph-excluded: true
46
47 ## 五、scripts/lint.py 检查项 (验证是否包含全部 9 项)
48
49 ## 六、qmd 状态
50 - qmd status 输出 (索引文件数量)
51 - 执行一次测试查询, 输出 top 3 结果
52
53 输出:  通过 /  未通过 (含缺失内容) / 建议修复优先级
```

Step 4: 配置 Obsidian Web Clipper

1. 安装 Obsidian Web Clipper 浏览器扩展
2. 创建「LLM Wiki - Article」模板:

Note location: `raw/clippings`

Note name: `{{data|date:YYYY-MM-DD}}-{{title}}`

Properties:

Property Name	Property Value
title	{{title}}
source	{{url}}
author	{{author}}
date	{{data\ date:YYYY-MM-DD}}
tags	["raw", "raw/clipping"]
processed	FALSE
Note content	{{content}}

- 3. 附件路径设为 `raw/images/`
- 4. 快捷键建议: `Ctrl+Shift+D`

LLM Wiki 使用文档

基于 Andrej Karpathy llm-wiki 模式的个人知识库系统。
核心理念：**你只负责剪裁，LLM 负责理解和沉淀。**

目录

- 1. 系统架构
- 2. 首次安装
- 3. 日常工作流
- 4. 操作详解
- 5. Obsidian 集成
- 6. 文件约定
- 7. Frontmatter 规范
- 8. Confidence 与来源完整性
- 9. 常见问题

1. 系统架构

三层模型



核心设计原则

原则	含义
LLM 是编译器，不是检索器	知识被「编译」一次后持久存在，而非每次查询重新推导
Raw 层不可变	原始来源绝对不会被 LLM 修改，永远是你的真实记录
Raw 层完整性由 SHA-256 守护	每次摄入记录哈希，lint 时检测文件是否被篡改
输出必须持久化	每次有价值的答案都写入 wiki/outputs/，不会消失在对话历史中
矛盾必须显式标注	来源之间的分歧会被明确记录，不会静默覆盖
认知演化可追踪	每个 concept 页的 Evolution Log 记录了认知如何随来源积累而演变
个人立场独立存储	自己写的文章不参与 confidence 计数，立场标注为「第一手认知」
confidence: high 必须用户确认	不能自动晋升，必须用户明确背书

2. 首次安装

前提条件

- 已安装 Claude Code (`npm install -g @anthropic-ai/claude-code` 或参考官方文档)
- Python 3.8+ (用于 lint 脚本)
- 推荐安装 Obsidian (用于浏览 Wiki)

步骤一：初始化项目

代码块 创建知识库目录

```
2  mkdir -p ~/knowledge-base
3  cd ~/knowledge-base
4
5  # 打开 Claude Code
6  claude
```

步骤二：执行提示词

将「给 Claude Code 的提示词」中的全部内容复制，粘贴到 Claude Code 对话框中，回车执行。
等待 Claude Code 完成全部初始化任务（约 2-5 分钟）。

步骤三：用 Obsidian 打开

1. 打开 Obsidian → Add vault → Open folder as vault
2. 选择 `~/knowledge-base`
3. 推荐安装插件：
 - **Obsidian Web Clipper**（剪裁文章）
 - **Dataview**（查看 frontmatter 统计）
 - **Marp for Obsidian**（预览幻灯片输出）

步骤四：配置 Web Clipper

1. 安装 Obsidian Web Clipper 浏览器扩展
2. 设置默认保存路径为 `raw/clippings/`
3. 设置附件路径为 `raw/images/`
4. 快捷键建议： `Ctrl+Shift+D`

3. 日常工作流

推荐节奏

频率	操作	说明
每天	剪藏文章到 raw/	Web Clipper 保存到 clippings/, 截图保存到 images/
每周	ingest 新来源	1-2 篇, 逐一处理而非批量
每两周	lint	运行健康检查, 修复 broken links 和 stub 页面
每月	reflect	二阶合成, 发现跨来源模式, 更新健康度仪表盘
随时	add-question	有疑问时立即记录到 QUESTIONS.md
随时	query	提问, 答案自动持久化到 outputs/

你的日常（每天 5 分钟）

代码块

```
1  1. 阅读文章 → Web Clipper 保存到 raw/clippings/
2  2. 重要截图 → 保存到 raw/images/
3  3. 随手想法 → 直接创建文件放入 raw/notes/
4  4. 自己的文章/分析 → 放入 raw/personal/
```

LLM 的日常

代码块

```
1  # 场景 1: 处理一篇新文章 (外部来源)
2  你: ingest raw/clippings/some-article.md
3  LLM: (计算哈希 → 创建摘要 → 更新概念页 → 写 Evolution Log → 检查 QUESTIONS → 写入 log)
4
5  # 场景 2: 处理自己写的文章 (个人写作)
6  你: ingest raw/personal/my-investment-note.md
7  LLM: (识别为 personal-writing → 提取立场 → 写入 My Position 节 → Evolution Log 记录)
```

```
8
9 # 场景 3: 查询知识
10 你: 根据我的知识库, 解释一下注意力机制的核心思想
11 LLM: (溯源到 source 页 → 合成答案 → 写 Confidence Notes → 保存到 wiki/outputs/)
12
13 # 场景 4: 记录开放问题
14 你: 我想搞清楚长期定投和一次性投入哪个更适合我
15 LLM: (规范化问题 → 追加到 wiki/QUESTIONS.md → 记录到 log)
16
17 # 场景 5: 健康检查
18 你: lint
19 LLM: (运行 lint.py → 报告 broken links / stubs / 哈希变化 / stale 页面 → 询问是否修复)
20
21 # 场景 6: 发现规律
22 你: reflect
23 LLM: (反向检验 → 模式扫描 → 深度合成 → Gap Analysis → 更新仪表盘)
```

4. 操作详解

4.1 INGEST — 摄入

触发词: `ingest`、`摄入`、`处理这个`

外部来源流程 (raw/articles/、raw/clippings/、raw/pdfs/) :

代码块

```
1 ingest raw/articles/my-article.md
```

LLM 会按顺序执行:

1. 计算原始文件的 SHA-256 哈希
2. 与你确认核心要点
3. 创建 `wiki/sources/<slug>.md`, frontmatter 中写入 `raw_sha256` 和 `last_verified`
4. 提取概念和实体, 创建/更新 `wiki/concepts/` 和 `wiki/entities/` 页面
5. 更新 concept 页的 Evolution Log (强化/修正/分歧)
6. 若来源超过 2 年前, 标注 `possibly_outdated: true`
7. 检查 `wiki/QUESTIONS.md`, 若有来源能回答的问题则提示你
8. 更新 `wiki/index.md`

9. 在 `wiki/log.md` 追加记录

个人写作流程 (raw/personal/) :

代码块

```
1 ingest raw/personal/my-investment-analysis.md
```

LLM 会：

- 不生成 Summary 节，跳过客观摘要
- 将核心论点提取后写入相关 concept 页的 `## My Position` 节（标注「第一手认知」）
- 在 Evolution Log 记录：「YYYY-MM-DD 个人写作 slug 确立了对此概念的明确立场」
- **不参与 confidence 的 source_count 计数**（避免用自己的文章给自己背书）
- 若文章引用了外部来源，尝试建立 wikilinks

预期耗时：每篇文章 1-3 分钟。建议前 5 篇逐一处理确认质量后再批量。

4.2 QUERY — 查询

触发词：直接提问即可，或说「根据我的知识库」

示例：

代码块

```
1  根据我的知识库，什么是最重要的投资原则？
2  帮我对比 A 和 B 这两个概念
3  把关于 [主题] 的主要观点整理成幻灯片
```

LLM 的执行步骤：

1. 读取 `wiki/index.md`，识别最相关的 3-5 个页面
2. 读取这些页面
3. **每个核心结论必须追溯到具体的 `wiki/sources/slug` 页面**（不允许只引用 concept 页）
4. 将答案写入 `wiki/outputs/<slug>-YYYY-MM-DD.md`
5. **在输出文件末尾必须包含「`△ Confidence Notes`」节**，列出所有 low/medium confidence 的引用
6. 更新 `wiki/index.md` 的 Recent Synthesis 列表
7. 在 `wiki/log.md` 追加记录

输出格式（根据问题类型）：

- 普通问题 → Markdown 正文
- 比较类问题 → Markdown 表格
- 演示类问题 → Marp 幻灯片（frontmatter 加 `marp: true`）
- 趋势类问题 → Python matplotlib 代码块
- 清单类问题 → 结构化 bullet list

4.3 LINT — 健康检查

触发词：`lint`、`检查`、`健康检查`

代码块

1 你: lint

lint.py 检查项：

#	检查项	说明
1	Frontmatter 完整性	所有 .md 文件是否有合法 YAML frontmatter（含 type 和 date）
2	Broken Wikilinks	[[xxx]] 引用了不存在的页面
3	Index 一致性	index.md 中标记为 processed 的来源是否有对应文件
4	Stub 页面	正文少于 100 字的空壳页面
5	SHA-256 完整性	raw 文件哈希与 source 页记录的哈希是否一致（⚠ SOURCE MODIFIED）
6	Stale 页面	超过时效阈值的 concept 页面（根据 domain_volatility）

Staleness 时效阈值：

- domain_volatility: high → 90 天
- domain_volatility: medium → 180 天
- domain_volatility: low → 365 天

输出： `wiki/outputs/lint-YYYY-MM-DD.md` 报告文件，同时终端打印摘要。

建议频率：每两周一次。

4.4 REFLECT — 二阶合成

触发词： `reflect`、 综合分析、 发现规律

四阶段执行：

Stage 0（反向检验）：在生成任何合成结论之前，主动在已有 sources 中搜索与候选结论相矛盾的证据。若找不到反对声音，在 Limitations 节标注：「⚠ 回音室风险：未找到反驳来源，结论可能存在确认偏差」

Stage 1（模式扫描）：仅扫描 index.md，识别跨来源的模式、隐性关联、内容空白、可能的矛盾对。

Stage 2（深度合成）：对有证据支撑的候选项，写入 `wiki/synthesis/<topic>-synthesis.md`。

Stage 3（Gap Analysis）：

- 找出 source_count = 1 且创建超过 30 天的页面（孤立概念）
- 找出多个 sources 提及但尚无独立页面的概念/实体（隐性盲区）
- 找出知识库覆盖明显稀薄的主题领域
- 输出到 `wiki/outputs/gap-report-YYYY-MM-DD.md`

完成后：更新 `wiki/overview.md` 的 Health Dashboard，更新 `wiki/index.md` 的 Recent Synthesis，在 `wiki/log.md` 追加记录。

建议频率：每月一次，或每积累 10 篇新来源后一次。

4.5 ADD-QUESTION — 记录问题

触发词： 我想搞清楚、 `add question`、 记录一个问题

示例：

代码块

1 你：我想搞清楚价值投资和成长投资的核心区别是什么

执行步骤：

- 1. 将问题规范化（提取核心疑问）
- 2. 追加到 `wiki/QUESTIONS.md` 的 Open Questions：

代码块

```
1 - [ ] 价值投资和成长投资的核心区别是什么 (opened YYYY-MM-DD)
```

- 1. 在 `wiki/log.md` 追加记录

当 `ingest` 时，LLM 会检查新来源是否能回答开放问题，若是则提示你。

4.6 MERGE — 去重合并

触发词：`merge`、`去重`

典型场景：

- `wiki/concepts/attention.md` 和 `wiki/concepts/attention-mechanism.md` 内容重叠
- Lint 报告发现大量指向同一主题的不同名称页面

执行前 LLM 会先与你确认合并方案，不会自动合并。合并后更新所有指向被合并页面的 wikilinks。

5. Obsidian 集成

推荐插件

插件	用途	安装方式
Obsidian Web Clipper	浏览器剪藏到 raw/	浏览器扩展商店
Dataview	查询 frontmatter, 显示统计	Obsidian 社区插件
Marp for Obsidian	预览 LLM 生成的幻灯片	Obsidian 社区插件
Graph View (内置)	可视化 Wiki 概念关系	无需安装

Dataview 查询示例

在 Obsidian 中创建一个 Dashboard 页面：

概念置信度排行：

代码块

```
1   ```dataview
2   TABLE date, source_count, confidence, domain_volatility
3   FROM "wiki/concepts"
4   SORT source_count DESC
5   LIMIT 20
6   ```
```

待处理来源列表：

代码块

```
1   ```dataview
2   LIST
3   FROM "wiki/sources"
4   WHERE processed = false
5   SORT date DESC
6   ```
```

Stale 概念检查（domain_volatility = high）：

代码块

```
1   ```dataview
2   TABLE date, last_reviewed, source_count, confidence
3   FROM "wiki/concepts"
4   WHERE domain_volatility = "high"
5   SORT last_reviewed ASC
6   ```
```

Personal Writings 列表：

代码块

```
1   ```dataview
2   TABLE date, status, confidence_at_writing
3   FROM "wiki/sources"
4   WHERE type = "personal-writing"
5   SORT date DESC
```

开放问题数：

代码块

```

1  ```dataview
2  TABLE date
3  FROM "wiki/QUESTIONS.md"
4  WHERE file.type = "questions"
5  FLATTEN file.lists AS list
6  WHERE list.checked = false
7  ```

```

Wikilinks 图谱

打开 Obsidian 的 Graph View (`Ctrl+G`) ，你可以看到：

- 大节点 = 被多个来源引用的核心概念
- 来源页面围绕概念聚合
- 合成页面 (synthesis) 连接多个概念
- Stale 页面可用不同颜色区分 (需 Dataview 插件)

6. 文件约定

完整目录结构

代码块

```

1  knowledge-base/
2  |—— raw/                                # 你的原始来源 (只增不改)
3  |   |—— articles/                       # 长文章 (Markdown)
4  |   |—— clippings/                     # Web Clipper 剪藏
5  |   |—— images/                        # 截图、图表
6  |   |—— pdfs/                          # PDF 文件
7  |   |—— notes/                         # 随手记录
8  |   |—— personal/                     # ★ 自己写的完整文章、分析报告、投资笔记
9  |—— wiki/                             # LLM 维护
10 |   |—— index.md                       # ★ LLM 读取的第一个文件
11 |   |—— log.md                         # 操作日志 (只追加)
12 |   |—— overview.md                   # 高层综述 + ★ Knowledge Base Health Dashboard
13 |   |—— QUESTIONS.md                  # ★ 开放问题队列
14 |   |—— sources/                      # 每个来源的摘要页 (外部来源 + 个人写作)

```


15		—	concepts/	# 概念/思想/模式 (含 Evolution Log)
16		—	entities/	# 人物/工具/机构/论文
17		—	synthesis/	# 跨来源合成分析
18		—	outputs/	# 查询答案、幻灯片、图表、lint 报告
19		—	templates/	# 页面模板 (LLM 使用)
20		—	scripts/	
21		—	lint.py	# ★ 健康检查脚本 (含 SHA-256 和 Stale 检查)
22		—	BOOTSTRAP_PROMPT.md	# 初始化提示词
23		—	UPGRADE_PROMPT.md	# ★ 系统升级提示词
24		—	CLAUDE.md	# ★ LLM 行为契约 (核心文件)
25		—	README.md	# 项目说明

7. Frontmatter 规范

所有页面必须包含

代码块

```

1  ---
2  type: <page-type>
3  title: "页面标题"
4  date: YYYY-MM-DD
5  tags: [wiki, wiki/<type>]
6  ---

```

Source 页 (外部来源) 额外字段

代码块

```

1  ---
2  type: source-summary
3  title: "{{title}}"
4  date: YYYY-MM-DD
5  source_url: "{{url}}"
6  domain: "{{domain}}"
7  author: "{{author}}"
8  tags: [wiki, wiki/source]
9  processed: true
10 raw_file: "raw/articles/filename.md" # 相对路径
11 raw_sha256: "<64-char-hex>" # SHA-256 哈希
12 last_verified: YYYY-MM-DD # 最近一次验证哈希的日期
13 possibly_outdated: false # 来源是否超过 2 年

```

Source 页（个人写作）额外字段

代码块

```

1  ---
2  type: personal-writing
3  title: "{{title}}"
4  date: YYYY-MM-DD           # 写作日期（非摄入日期）
5  status: draft              # draft / published / deprecated
6  topic_tags: []
7  confidence_at_writing: medium # 写作时对内容的把握程度
8  superseded_by: ""         # 若有更新的文章，填入新文章路径
9  raw_file: "raw/personal/filename.md"
10 raw_sha256: "<hash>"
11 last_verified: YYYY-MM-DD
12 tags: [wiki, wiki/source]
13 processed: true
14 ---

```

Concept 页额外字段

代码块

```

1  ---
2  type: concept
3  title: "{{title}}"
4  date: YYYY-MM-DD
5  updated: YYYY-MM-DD
6  tags: [wiki, wiki/concept]
7  source_count: 0           # 引用此概念的外部来源数量
8  confidence: low           # low / medium / high
9  domain_volatility: medium # low / medium / high
10 last_reviewed: YYYY-MM-DD # 最近一次更新此概念的日期
11 ---

```

Confidence 更新规则

来源数量	Confidence 值	说明
1 个来源	low	自动
3+ 个来源	medium	自动
5+ 个来源且无重大矛盾	候选 high	禁止自动晋升
用户确认后	high	必须用户明确回复「确认」或「ok」

注意：个人写作（raw/personal/）不参与 source_count 计数。

Evolution Log 格式

代码块

```
1  ## Evolution Log
2
3  <!-- 每次更新此概念页时追加一条 -->- 2026-04-06 (2 sources) : 强化: 来自
   [[wiki/sources/attention-paper]] 和 [[wiki/sources/transformer-blog]] 的证据一致
4  - 2026-04-10 (3 sources) : 修正: 新增 [[wiki/sources/new-source]] 显示原定义忽略了
   X 维度
```

8. Confidence 与来源完整性

为什么 confidence: high 不能自动晋升？

系统升级前，5 个来源就能让一个概念自动变成「高置信度」。但这会产生错误复利：错误的概念一旦被标记为 high，后续 query 就会大量引用它，形成自我强化。

升级后的机制：

- 达到 5 个来源 → 进入「候选 high」状态
- LLM 必须向你展示该 concept 页的完整 Definition 和 Sources 列表
- 你明确回复「确认」后，confidence 才变为 high
- high 是你的**主动背书**，不是计数器的输出

如何判断是否确认 high？

LLM 会问：「概念 X 现在有 5+ 个来源且无矛盾，是否确认 confidence 为 high？」你需要回顾：

- 这些来源的权威性如何？

- 定义是否经过你的审查和认可？
- 你自己是否认同这个定义？

只有在你真正理解和认可时才能确认。

SHA-256 完整性守护

每次 ingest 时，LLM 计算原始文件的 SHA-256 哈希并写入 source 页 frontmatter。Lint 时会重新计算并比对。如果 raw 文件在你不知情的情况下被修改（哪怕是一个字符），lint 就会报告 ⚠ SOURCE MODIFIED，提示你重新摄入。

9. 常见问题

Q：我编辑了 raw 文件，LLM 会知道吗？

不会主动知道。但下次运行 `lint` 时，`lint.py` 会检测到 SHA-256 哈希变化，报告 ⚠ SOURCE MODIFIED，并建议你重新摄入。重新摄入后会更新所有受影响的 concept/entity 页面，并在 Evolution Log 记录变更。

Q：如何处理自己写的旧文章？

将文章放入 `raw/personal/` 目录，然后用 `ingest raw/personal/你的文章.md` 处理。LLM 会识别为 personal-writing 来源：

- 提取核心论点写入相关 concept 页的 `## My Position` 节
- 在 Evolution Log 记录立场确立
- 不参与 confidence 计数（避免用自己的文章给自己背书）

Q：confidence: high 是怎么产生的？

不能自动产生。当一个 concept 达到 5+ 个外部来源且无重大矛盾时，LLM 会暂停并请求你确认。你明确回复「确认」或「ok」后，confidence 才更新为 high。这是你对知识库内容的主动背书，而非系统自动判断。

Q：如何查看知识库健康状态？

查看 `wiki/overview.md` 的 Knowledge Base Health Dashboard，每次执行 REFLECT 后自动更新。包含：总来源数、高置信度概念数、开放问题数、Stale 页面数、孤立概念数、知识密集/空白领域等指标。

Q：如何避免错误复利？

1. **Confidence 门控**：high 必须你确认，不能自动晋升
2. **溯源到 source**：每个核心结论必须追溯到具体 source 页，不允许只引用 concept

- 3. 回音室风险检查：REFLECT Stage 0 会主动搜索反驳证据
- 4. 矛盾显式标注：Evolution Log 记录认知变化，Contradictions 节记录分歧
- 5. Staleness 衰减：长期未更新的概念会被标记，避免过时内容自我强化

Q：来源超过 2 年会怎样？

LLM 会在 source 页的 frontmatter 标注 `possibly_outdated: true`，并在摘要末尾添加提示。这不是说你不能用它，而是提醒你在基于该来源做决策时要额外谨慎。

Q：LLM 会不会用旧来源的答案回答新问题？

不会。每次 QUERY 时 LLM 会先读取 index.md 识别相关页面，然后综合多个来源的证据合成答案。系统设计鼓励不断用新来源更新已有概念页（同时更新 last_reviewed 和 Evolution Log），而不是让旧内容持续占据主导。

Q：CLAUDE.md 需要我自己维护吗？

CLAUDE.md 会随你的使用自然演化。当你发现 LLM 的某类输出不符合期望时，告诉它「请更新 CLAUDE.md，以后处理 [情况] 时要 [规范]」，让 LLM 自己更新行为契约。

附录：与 RAG 的根本区别

维度	传统 RAG	LLM Wiki（本项目）
知识编译时机	每次查询时实时推导	摄入时一次编译，永久存储
知识积累	无积累，每次独立	复利式增长，来源越多洞察越深
矛盾处理	向量检索忽略矛盾	显式标注来源分歧
查询结果	仅出现在对话中	写入 wiki/outputs/，永久保存
知识导航	向量相似度	目录导航 + 人工策划的 wikilinks
认知演化	不可追踪	Evolution Log 全程记录
错误复利防护	无	Confidence 门控 + 溯源机制 + 反向检验
你的投入	每次查询都需构建 prompt	一次摄入，后续复用

